
IPC2_PROYECTO 3_202210484

Resumen

El Sistema de Autorización de Documentos Tributarios Electrónicos (DTE) se desarrolló como una herramienta destinada a la Superintendencia de Administración Tributaria (SAT), con el propósito de facilitar la gestión y validación legal de los documentos fiscales digitales emitidos por los contribuyentes. Esta plataforma tecnológica recibe solicitudes estructuradas en formato XML, valida automáticamente la información, verifica los cálculos fiscales y se asegura de que cumpla con la normativa vigente, devolviendo una respuesta con un código único de autorización, el cual respalda el documento ante la SAT.

El sistema se estructura con una arquitectura cliente-servidor. El backend, desarrollado en Flask, realiza las validaciones necesarias y contiene la lógica de funcionamiento, mientras que el frontend, implementado con Django, ofrece una interfaz amigable que permite a los usuarios cargar archivos, verificar resultados y generar reportes. Ambos módulos se comunican mediante una API REST, permitiendo escalabilidad y facilidad de mantenimiento.

Palabras clave

DTE (Documento Tributario Electrónico): Documento digital emitido para fines fiscales, validado por la SAT.

NIT (Número de Identificación Tributaria): Código único para contribuyentes, validado mediante algoritmo específico.

API RESTful: Interfaz de comunicación entre frontend y backend usando protocolo HTTP y métodos estándar (GET, POST).

Abstract

The Electronic Tax Document Authorization System (DTE) was developed as a tool for the Superintendency of Tax Administration (SAT) to facilitate the management and legal validation of digital tax documents issued by taxpayers. This technological platform receives structured requests in XML format, automatically validates the information, verifies tax calculations, and ensures compliance with current regulations. It returns a response with a unique authorization code, which validates the document with the SAT.

The system is structured with a client-server architecture. The backend, developed in Flask, performs the necessary validations and contains the operating logic, while the frontend, implemented with Django, offers a user-friendly interface that allows users to upload files, verify results, and generate reports. Both modules communicate through a REST API, allowing scalability and ease of maintenance.

Keywords

DTE (Electronic Tax Document): Digital document issued for tax purposes, validated by the SAT.

NIT (Tax Identification Number): Unique code for taxpayers, validated using a specific algorithm.

RESTful API: Communication interface between the frontend and backend using the HTTP protocol and standard methods (GET, POST).

Introducción

Con la evolución digital que atraviesan las instituciones públicas, la Superintendencia de Administración Tributaria (SAT) ha implementado mecanismos electrónicos que mejoran la gestión de documentos fiscales. En ese contexto, se ha desarrollado un sistema que permite a los contribuyentes generar y enviar Documentos Tributarios Electrónicos (DTE) para su autorización de forma automatizada.

Este proyecto tiene como objetivo principal construir una solución tecnológica que permita procesar solicitudes XML, verificar su validez legal y devolver un resultado que indique si el documento puede ser aprobado. Para ello, se emplean tecnologías como Flask y Django, organizadas bajo un modelo cliente-servidor. Este ensayo describe las etapas del desarrollo, las herramientas utilizadas y los beneficios que ofrece el sistema, tanto para los contribuyentes como para la administración tributaria.

Desarrollo del Tema

El sistema propuesto funciona como una herramienta que automatiza la validación de Documentos Tributarios Electrónicos (DTE) enviados a través de archivos XML. El proceso inicia cuando el usuario carga el archivo correspondiente, el cual es procesado mediante una API desarrollada en Flask. Esta API se encarga de interpretar la estructura del documento, aplicar reglas de validación, verificar cálculos tributarios y generar una respuesta formal.

En la etapa de validación, se implementan estructuras de datos como pilas y listas enlazadas para organizar internamente los datos del XML. Esto permite recorrer de manera eficiente los nodos

del documento y evaluar si cumplen con los requisitos establecidos. Además, se analiza la consistencia de los totales y subtotales, con el objetivo de asegurar que no existan errores en el cálculo de impuestos.

Por otro lado, el sistema incluye un componente visual desarrollado en Django, que permite al usuario interactuar con el sistema a través de una interfaz sencilla. Desde allí se pueden cargar archivos, consultar el estado de los documentos y visualizar representaciones gráficas generadas con Graphviz. Estas gráficas muestran el flujo interno del procesamiento del documento, lo cual resulta útil para tareas de depuración o explicación técnica.

La comunicación entre el cliente y el servidor se realiza mediante una API REST, permitiendo que ambos componentes funcionen de forma desacoplada y puedan ser escalados o reemplazados independientemente. Esto favorece la adaptabilidad del sistema a diferentes contextos o a futuros requerimientos tecnológicos.

1.1 Programación Orientada a Objetos

El sistema está implementado siguiendo el paradigma de Programación Orientada a Objetos (POO), lo que permite:

- **Encapsulamiento:** Los datos y métodos relacionados están encapsulados en clases, ocultando los detalles de implementación y exponiendo solo las interfaces necesarias.
- **Herencia:** Permite la reutilización de código a través de relaciones jerárquicas entre clases.
- **Polimorfismo:** Facilita el tratamiento uniforme de diferentes tipos de objetos.

- **Abstracción:** Permite modelar entidades del mundo real de manera clara y concisa.

1.2 Arquitectura Cliente-Servidor

El sistema implementa una arquitectura cliente-servidor donde:

- El **Backend (Flask)** actúa como servidor, procesando solicitudes y proporcionando servicios a través de una API RESTful.
- El **Frontend (Django)** actúa como cliente, proporcionando una interfaz de usuario intuitiva y comunicándose con el backend a través de HTTP.

1.3 Arquitectura por Capas

Tanto el backend como el frontend siguen una arquitectura por capas:

- **Capa de Presentación:** Interfaz de usuario (Frontend) y API (Backend).
- **Capa de Lógica de Negocio:** Procesamiento y validación de datos.
- **Capa de Acceso a Datos:** Almacenamiento y recuperación de información.

1.4 Validación de NIT

El algoritmo para la validación del NIT (Número de Identificación Tributaria) es un componente crítico del sistema:

1. Multiplicar cada dígito por su posición (de derecha a izquierda, comenzando desde 2).
 2. Sumar todos los productos.
 3. Calcular el módulo 11 de la suma.
 4. Restar el resultado del paso 3 de 11.
 5. Calcular el módulo 11 del resultado del paso 4.
- Si el resultado es 10, el dígito verificador debe ser 'K'.

- De lo contrario, el dígito verificador debe ser el número resultante.

Funcionalidades del Sistema

1. Backend (Flask)

1.1 Procesamiento de Solicitudes

- Recibe solicitudes de autorización en formato XML.
- Parsea y valida la estructura del XML.
- Extrae la información de cada DTE.

1.2 Validaciones

- Validación de NIT emisor e NIT receptor utilizando el algoritmo especificado.
- Validación del cálculo de IVA ($\text{VALOR} * 0.12$, redondeado a 2 decimales).
- Validación del cálculo del TOTAL ($\text{VALOR} + \text{IVA}$).
- Detección de referencias duplicadas por fecha.

1.3 Generación de Autorizaciones

- Creación de códigos de autorización con formato `yyyymmdd#####`.
- Gestión de correlativos diarios que reinician en 1 cada día.

1.4 Almacenamiento de Datos

- Generación del archivo XML de salida (autorizaciones.xml).
- Mantenimiento de estadísticas por fecha.

1.5 Generación de Reportes

- Resumen de IVA por fecha y NIT.
- Resumen por rango de fechas.
- Generación de gráficas estadísticas.
- Exportación de reportes en formato PDF.

2. Frontend (Django)

2.1 Interfaz de Usuario

- Pantalla principal con información del sistema.
- Formulario para cargar archivos XML.
- Visualización de resultados y mensajes de error.

2.2 Gestión de Archivos

- Carga de archivos XML con solicitudes de autorización.
- Envío de archivos al backend para procesamiento.

2.3 Visualización de Datos

- Consulta del archivo de autorizaciones.
- Visualización de resúmenes y estadísticas.
- Visualización de gráficas interactivas.

2.4 Generación de Reportes

- Generación de reportes en formato PDF.
- Exportación de datos y gráficas.

2.5 Ayuda y Documentación

- Información del estudiante.
- Documentación del sistema.

solución robusta y flexible. Asimismo, la integración de estructuras de datos y diagramas generados con Graphviz aportó claridad tanto en la organización interna del sistema como en la visualización de los procesos ejecutados.

En resumen, el proyecto demuestra cómo el uso de tecnologías modernas puede ser aplicado en la resolución de problemas reales dentro del ámbito fiscal, brindando soluciones eficientes que beneficien tanto a los contribuyentes como a las instituciones públicas encargadas de la recaudación.

Conclusiones

El desarrollo de este sistema de autorización de Documentos Tributarios Electrónicos representa una contribución significativa a la digitalización de los procesos fiscales en Guatemala. Al automatizar la validación de documentos XML conforme a las normativas de la SAT, se logra no solo agilizar el proceso, sino también garantizar mayor precisión y seguridad en el manejo de la información tributaria.

El uso de herramientas como Flask para el backend y Django en el frontend permitió implementar una